

The Ultimate JARVIS Systems Architecture Synthesis

Reverse-engineering 18+ Jarvis-style AI assistant builds into one optimal blueprint.

Evidence labels

VERIFIED = independently confirmed from code, repos, docs, or primary sources.

INFERRED = consistent with available evidence but not directly confirmed.

CLAIMED = asserted by the creator with no independent support.

How this was researched

Two prior deep-dive reports (7 Instagram reels; 13 YouTube builds - each researched from video descriptions, chapters, GitHub repos with one repo read file-by-file, Instagram captions/comments/follow-ups, and independent verification of every named component and price) were re-read in full. Eight additional systems and projects from a 70-video discovery sweep were then researched fresh from primary sources - the OpenJarvis local-Qwen video build, eadmin2's Hermes command center, Thanhly's LiveKit+Gemini voice tutorial, James Goldbach's "Donna" Telegram life-OS, Qendresa Hoti's "Life Admin Brain" RAG, the memU memory project, Stanford's OpenJarvis paper/repo, and OpenClaw - with repo READMEs and in several cases source files read directly. Pure funnels and vaporware were excluded from profiling (reasons noted); everything else got a full technical teardown.

PHASE 1 - Individual system profiles

Profile 1 - Zubair / AI Workshop: JARVIS on Claude Fable 5.1 + Claude Code

Evidence: components VERIFIED · wiring INFERRED · build prompts paywalled

End-to-end workflow. The real engine is Claude Code as the *builder*, not the assistant. Eight prompts generate a local app that becomes the runtime: (1) microphone -> Web Speech API speech recognition (free, Chrome) -> text; (2) backend sends text + tool definitions to an LLM endpoint - Anthropic API with Claude Fable 5.1 by default; (3) the "brain hot-swap" is switching the API base URL and model string (e.g., to OpenRouter for Grok/GPT) - a config change, not a transplant; (4) a folder of markdown notes is parsed into a local embedding index, queried semantically, and rendered as a three.js-style 3D "knowledge galaxy" the view can fly to for provenance; (5) webcam/screen frames are sent to the multimodal endpoint as vision input; (6) replies spoken via free browser speech synthesis; (7) Gmail/Calendar via Google OAuth; (8) outbound phone calls are outsourced to **Retell AI** (cloud voice-agent + telephony), not his code; (9) the "self-verification harness" is a generated test script running canned checks.

Tech stack. Inference: Claude Fable 5.1 via Anthropic API (VERIFIED real release Sept 1 2026: 1M-token context; \$10/M input, \$50/M output, cache reads \$0.25/M). Orchestration: generated app code around tool-calling. STT/TTS: Web Speech API both directions (free, mediocre - no reliable wake word or barge-in, Chrome-dependent). Vision: frames -> multimodal model (token-hungry). Memory: markdown notes -> local embedding index (exact store undisclosed). Tools: Google OAuth (Gmail/Calendar - the biggest real friction), Retell AI (~\$0.11-0.16/min + \$2/mo number). Hardware: his Mac; PC/Mac compatible. No public repo.

Strengths. Most capable *shown* architecture; the cited-answer provenance habit ("show me the exact note this came from") is genuinely good engineering; hot-swap proves the model-is-config thesis.

Limitations. Prompts gated behind Skool paywall - "one prompt does the work of three" unverifiable, no independent reproductions found. Always-on frontier API tokens burn fast. Mac control needs accessibility permissions and breaks across OS updates.

Profile 2 - Hermes Agent as JARVIS runtime (Software Gent tutorial)

Evidence: runtime VERIFIED · tutorial's voice chain INFERRED · known fail-silent bugs VERIFIED

End-to-end workflow. Install Hermes (Python, MIT) on a machine or ~€5/mo VPS: CLI + messaging gateway (Telegram/Discord/WhatsApp) + agent loop handling provider selection, tool execution, context compression, session persistence. Model of choice: NVIDIA NIM free credits, OpenRouter, or local Ollama/LM Studio. Voice input = **Wispr Flow** dictation hotkey (hold key, speak, cloud transcribes, text typed into CLI) - dictation, not conversational voice. Voice output = ElevenLabs TTS API reading replies. The "command matrix" = saved prompt templates/skills/aliases. Memory = SQLite session DB with FTS5 full-text search + a learning loop where the agent writes Markdown SKILL.md procedure files injected into future prompts.

Tech stack. LLM: any (NVIDIA NIM / OpenRouter / Ollama / Anthropic / OpenAI) - genuinely model-agnostic. Tools: ~47 built-in (shell, files, browser, web search, MCP servers, cron, subagent delegation). STT: Wispr Flow (cloud; Pro \$15/mo, free tier 2,000 words/week). TTS: ElevenLabs API (metered; free ≈ 10 min/mo). No wake word, no barge-in, no phone calls - the Iron Man gap is large. Hosting: self-hosted; laptop asleep = JARVIS dead.

Strengths. The strongest *foundation* in the set: real, free, MIT, ~185k-244k GitHub stars, NVIDIA's reference runtime for Nemotron 3 Ultra. Only free runtime that is a genuine agent (tools + cron + memory + gateways), not a chatbot.

Limitations. VERIFIED structural issues (Apr 2026 code review): 10,700-line god-object agent file; "death spiral" where 212k+ token gateway sessions loop on API disconnects with no recovery; **silent history deletion on failed summarization**; silent skill-loading breakage. Dominant pattern: fail-silent on critical data operations - memory that deletes itself is worse than none. "Gets smarter with use" directionally true but oversold. Free path exists (OpenRouter :free 20 req/min, 50/day unfunded; Ollama local) but the video's shown config isn't free.

Profile 3 - Alex Schar: DIY capture-to-voice pipeline

Evidence: build VERIFIED (28+ part build-in-public, on-camera demos) · "Fletcher" retrieval INFERRED

End-to-end workflow. (1) Plaud recorder captures spoken life context continuously; (2) audio -> **LocalWhisper** (offline Mac transcription app, VERIFIED); (3) transcripts feed a large **Obsidian graph** - markdown notes linked in graph view that "sorts information for her agents"; (4) Hermes agents (maps cleanly to Nous Research's Hermes Agent - VERIFIED MIT, model-agnostic) run on a Mac Mini; her phrase "keeps the loop updated regardless of the AI model used" matches its documented design; (5) **"Fletcher"** - her own retrieval layer on the Mac Mini (also her dashboard/site name; no public repo - internals unverified); (6) Raspberry Pi 5 runs the interactive speech loop; M5Stack Atom Echo (\$13 ESP32 voice kit, VERIFIED) as listen/speak device; ceiling projector + overhead camera with hand-tracking for a projected desk dashboard (two Pis - one for hand-tracking, one for requests - after frame-rate drops while speaking); Amazon Echo for vocal playback; (7) Flipper Zero (VERIFIED) captures projector IR codes, Pi replays them - "Fletcher, turn on the projector" works on camera; microSD -> USB SSD swap (AI HAT blocked the SD slot) for offline-LLM headroom; ArduCam upgrade for hand tracking.

Tech stack. Capture: Plaud NotePin (\$159; 300 free transcription min/mo - note: Plaud transcription is cloud). STT: LocalWhisper (offline). Memory: Obsidian vault + graph (file-based retrieval - fine at personal scale, degrades as notes grow; one commenter correctly suggested a vector DB). Agents: Hermes Agent on Mac Mini. Voice out: Amazon Echo (puts voice through Amazon's cloud despite "offline" language) + Atom Echo. Hardware: Mac Mini (~\$599+), Pi 5 + HATs + SSD + camera + projector + Flipper Zero (~\$150) + Plaud.

Strengths. Most transparent build in the set; honest about failures (frame drops, "really shitty" camera rig). The memory thesis - "intelligence = context + relevancy" - is the single best idea across all builds. The capture -> transcribe -> graph -> retrieve -> act pipeline is the practical implementation.

Limitations. Pi is the bottleneck (hand-tracking frame rate visibly drops while AI speaks; offline LLMs on Pi are weak - real inference lives on the Mac Mini). Always-on Plaud recording = battery, privacy, consent overhead (records other people). Fletcher's ranking mechanics are a black box. Several hundred dollars of hardware; she funds via supporter wishlist.

Profile 4 - Chloe / chloeshy.ai: business-ops JARVIS

Evidence: components VERIFIED · quoted metrics glossy (treat as illustration) · harness glue hidden

End-to-end workflow. (1) Dashboard vibe-coded in Claude/Claude Code ("5-10 minutes" - hides the real API-key/MCP/auth glue work); (2) agent harness connects Claude to hosted voice models for the speech loop; (3) **Kokoro-82M TTS** (VERIFIED: 82M-param open TTS, Apache 2.0, ~350MB, CPU-runnable, free); (4) **Playwright MCP** (Microsoft's official MCP server, VERIFIED) gives Claude browser control + vision over her Chrome; (5) **RevenueCat MCP** (official, mcp.revenuecat.ai, launched July 2025, 26 tools - VERIFIED) pulls app revenue/subscriber metrics; **Meta Ads MCPs** (ZuckerBot, Pipeboard/creatibi, Ryze - all wrapping the Meta Marketing API, VERIFIED) pull ad spend/creative performance; (6) Jarvis reports 7-day figures by voice - 2,459 downloads, \$4,289 revenue, \$475 ad spend, claimed 1.5 ROAS - identifies top/weakest creatives, recommends reviewing backend PRs and researching organic content. The GitHub link she distributes is Stanford + Lambda Labs' **OpenJarvis** ("Personal AI, On Personal Devices" - VERIFIED via arXiv); her build is substantially OpenJarvis wiring + commodity MCP servers - legitimate, not from-scratch invention.

Tech stack. Models: Claude (Fable 5 family per her hashtag). STT: hosted voice models (unspecified). TTS: Kokoro-82M. Agents: Claude + Claude Code harness. Tools: Playwright MCP, RevenueCat MCP, Meta Ads MCP. Memory: none named. Surfaces: dashboard + voice.

Strengths. Clearest real business-operations demo: live metrics -> analysis -> spoken brief -> recommended actions. Every named component independently exists. The key jump - from reporting to *operating* (proposing the next bottleneck fix) - is the pattern to steal.

Limitations. Math caveat: $\$4,289 \div \$475 \approx 9x$, not the stated 1.5 ROAS - treat figures as illustration. Playwright browser automation is flaky/slow; Claude Code screen-capture reading degrades reliability. Meta Ads MCPs with write tools can spend real money - confirmation model matters enormously; Meta Marketing API needs an approved app + OAuth refresh pain. RevenueCat only pays off with a real app. Comments were a fulfillment funnel, thin on technical debate.

Profile 5 - Nate Herkman: no-code voice stack (Lovable + ElevenLabs + n8n)

Evidence: integration pattern VERIFIED (n8n templates, independent repos) · honest about "no code"

End-to-end workflow. Lovable generates a React web app (Lovable Cloud) with an embedded ElevenLabs voice widget. ElevenLabs runs the realtime voice stack (its STT -> configured LLM -> ElevenLabs TTS, with interruption handling) streamed to the browser. Tool calling: custom tools defined in the ElevenLabs agent (name, description, JSON schema) point at an **n8n Webhook node** (POST). Per call: user speaks -> ElevenLabs LLM invokes tool -> POSTs parameters to n8n -> n8n workflow runs (AI agent node, APIs, 400+ integrations) -> "Respond to Webhook" returns result -> agent speaks it. n8n never touches audio - pure tool backend.

Tech stack. Voice: ElevenLabs Conversational AI (\$0.08/min + LLM passthrough; 30 min/day ≈ \$72/mo voice alone). Automation: n8n (Cloud Starter €24/mo, 2,500 executions - workflows **halt at cap, no overages**; self-hosted Community free/unlimited on a ~\$4-7/mo server, but you own uptime/SSL/updates). UI: Lovable (\$25/mo Pro; free 5 build credits/day). Memory: thin - in-session context + optional knowledge base; **no real cross-session memory** unless built. No computer control beyond n8n integrations; no proactivity (only acts when spoken to); browser tab - close it and JARVIS is gone.

Strengths. Most honest no-code product: the architecture genuinely hangs together and is independently documented. The ElevenLabs-custom-tool -> n8n-webhook pattern is the correct way to give a voice agent hands without code. Good *voice front-end*.

Limitations. ~\$55-75/mo floor; each tool call adds 2-10s of dead air; three chained vendors = three failure points. No persistent memory = a mouth, not a mind. Do not mistake it for the brain.

Profile 6 - manujademin: 100% offline Python JARVIS

Evidence: components VERIFIED · wiring INFERRED (no public repo) · honest framing

End-to-end workflow. The canonical offline voice loop: mic stream -> **openWakeWord** detects the wake phrase on CPU in milliseconds -> utterance captured -> **faster-whisper** transcribes locally -> POST to Ollama local API (localhost:11434, Llama 3.1, likely 8B) -> response -> **Piper TTS** (local ONNX neural voice) -> speakers -> back to wake-word listening. Self-described "basic wrapper model" - open-source tools stitched together, nothing trained from scratch. Accurate and refreshing.

Tech stack. Wake: openWakeWord. STT: faster-whisper. LLM: Ollama + Llama 3.1 8B (~5-6 GB RAM quantized; CPU-only = seconds-to-tens-of-seconds per response; modest 8GB+ VRAM GPU -> near real-time). TTS: Piper (robotic next to cloud TTS). Memory: none disclosed. Integrations: none. \$0 software; hardware is the real cost. "100% offline" holds at runtime (only first-run downloads need internet).

Strengths. Most honest YouTube build; genuinely reproducible \$0 stack; the privacy tier every other build lacks.

Limitations. A voice chatbot, not an agent: Llama 3.1 8B is weak at function calling - talks, doesn't act. Wake+Whisper+LLM+TTS chain stacks latency. No persistent memory without adding a vector DB yourself. STT errors cascade with no correction loop.

Profile 7 - Arcus (@avoshuu): local-first "Chief Executive AI Officer"

Evidence: CLAIMED/in-progress · consistent 2-month story · zero inspectable internals

End-to-end workflow (reconstructed from claims). Voice/text trigger on any surface -> local orchestrator ("proper reasoning harness," per comments) -> dispatches to Research / Developer / Browser subagents on the "Dynamic agent island" dashboard -> tool use: screen capture + vision ("can legit see what's on screen"), hypervisor control ("spin up VMs by voice"), smart-home/room control, booking system, email drafting, project creation "on autopilot" -> response via voice mini-notch overlay or proactive outbound call. Surfaces: desktop app, voice notch, smartwatch (ping/call), Discord server, phone.

Tech stack. Models: unspecified local models - insists "fully local (not Claude or ChatGPT)" but names no model, STT, TTS, or memory store. Agents: unnamed reasoning harness + three subagents. Hardware: "her own PC," "now building servers." No repo, not open source. A reply claiming "full-backed YC startup" is unverified; skeptics allege Claude wrapper.

Strengths. Boldest vision: one brain across every surface, fully local - closest to the end state. Role-separated agents and proactive contact are the right instincts. Surfaces-as-replaceable-clients is the correct architecture.

Limitations. Fully-local agentic multi-tool models need serious GPU hardware (\$1,500+ rig). Screen-vision + multi-agent loops burn context/latency budgets. Watch/phone gateways need push infra + always-on service. Proactive autonomous actions (VM spin-up, bookings) without guardrails become expensive mistakes. Closed source = no audit of the "local" claim.

Profile 8 - Azaris / @lukebuildsai: the managed 6am-call product

Evidence: product VERIFIED (live site, pricing) · the reel is a scripted ad (not a demo)

End-to-end workflow (product, not skit). Azaris (azaris.ai): hosted agent platform, **\$97/mo DIY, bring-your-own model key** (no token markup), cloud browser + persistent memory included, channels: Telegram, Slack, Discord, Teams, voice. **Approval cards** before big decisions. 1,000+ integration catalog + native connectors + cloud-browser fallback. Referral loop (\$10/mo off per referral, up to 5) explains the aggressive funnel. Suggestive but unconfirmed: GitHub org azaris-ai-dev/openclaw implies OpenClaw-based. The 6am-call reel itself: scripted dramatization, no logs, no architecture - evaluate the product, not the skit.

Tech stack. Hosted agent platform; BYOK models; cloud browser; persistent memory (typically vector-store RAG - verify before trusting); multi-channel messaging; approval cards.

Strengths. The *pattern* is real and stealable: nightly trigger -> read queues (inbox, threads, metrics, code) -> act within policy / draft the risky -> audit trail -> morning brief -> voice call with WhatsApp fallback. Approval cards are exactly the right primitive.

Limitations. \$97/mo + your tokens; always-on cloud-browser agents burn tokens fast and are flaky on login-walled sites (2FA, CAPTCHAs). "Persistent memory" claims need verification. Comments: ~99% keyword spam, zero answered technical questions, no independent reviews found.

Profile 9 - AIDEVGENIUS: no-code mobile JARVIS

Evidence: mechanism VERIFIED · zero agentic capability VERIFIED

End-to-end workflow. Single static HTML page in the phone browser. Google AI Studio (Gemini) = code generator (paste creator's prompt -> page code). Runtime: Web Speech API (webkitSpeechRecognition, Chrome Android - audio to Google's servers, needs internet) -> fetch() POST to OpenRouter chat-completions with your key + JARVIS system prompt -> speechSynthesis speaks reply. Tap mic -> wait -> hear reply. Best-supported inference: Gemini writes the code, OpenRouter supplies runtime inference (no Google Cloud billing). No tools, no memory, no actions, no wake word. **Security caveat:** OpenRouter key in plaintext JS - fine for personal on-device use, but anyone copying the file gets the key.

Tech stack. LLM: OpenRouter free models (20 req/min, 50/day unfunded; 1,000/day after \$10 lifetime purchase; free models rate-limit hard at peak). STT: Web Speech API (cloud). TTS: device speechSynthesis. Cost: ~\$0.

Strengths. Genuinely \$0, reproducible voice-chat page; proves the floor of what's possible.

Limitations. A voice chatbot demo, not an assistant: it can *talk about* doing things, never do them. "No coding" is brittle - generated Web Speech code commonly breaks (permissions, voice-loading races, pauses cutting recognition); debugging JS in a phone text editor is miserable. Tab dies when the screen sleeps; no background operation.

Profile 10 - Z.E.R.O (@jonathonmj): the operator framing

Evidence: CLAIMED system · VERIFIED good framing · monetization = consulting calls

No verifiable system behind the theatrical brain UI (Motor Cortex / Prefrontal / Hippocampus nodes, "32 tools," parallel agents). Web search returns nothing relevant; "open source repo?" and "guide dropping soon" never materialized; #claudecode tags suggest Claude Code + MCP at most. The risk is **cargo-culting**: copying the aesthetic without idempotent tools, approval gates, memory hygiene, eval loops. Included because the *framing* is the correct mental model and becomes the Ultimate's operator contract: an LLM is a tool; JARVIS is an **operator** with purpose, tools, memory, permissions, and feedback loops. Tool vs. operator: a tool waits for a prompt and returns content; an operator wakes on events/schedules, pursues defined outcomes, runs multi-step state machines, observes and verifies, corrects or escalates, and remembers.

Profile 11 - Thanhly / ruxakK: LiveKit + Gemini realtime voice agent

Evidence: pipeline VERIFIED (agent.py + repo README read) · "100% free" CLAIMED (free-tier only) · tutorial repo identity INFERRED

End-to-end workflow. (1) Mic capture on any client - Next.js web app, Flutter mobile app, or terminal - publishes audio to a **LiveKit room** over WebRTC. (2) Noise cancellation first: AiCoustics `QUAIL_VF_S` enhancement on the audio track. (3) `AgentServer()` + `@server.rtc_session(agent_name="my-agent")` registers the Python worker; LiveKit dispatches the session when a user joins. (4) Audio + camera frames stream **directly into Gemini Live** (`gemini-3.1-flash-live-preview`, voice "Enceladus", en-GB) - a speech-to-speech realtime model, *not* a Deepgram/Cartesia STT->LLM->TTS cascade (the classic `stt` / `tts` params are commented out in the code). (5) Turn-taking: LiveKit's hosted `TurnDetector()` + adaptive interruption (distinguishes real interruptions from backchannel "mhm") + preemptive generation - layered client-side over Gemini Live's own server VAD, a fragile combo the docs flag (expect tuning). (6) When Gemini emits a function call, the LiveKit SDK routes it to `@function_tool()`-registered Python methods (schema from type hints + docstring). (7) 11 browser tools via a custom `BrowserManager`: headless Playwright Chromium, `open_url`, `search_the_web` (DuckDuckGo), `read_page`, `click`, `type_text`, screenshots, and `confirm_browser_action` - a safety gate requiring user confirmation before send/submit/purchase/delete. (8) Gemini speaks the result back into the room to whichever client is listening. All frontends (React 19 web HUD, Flutter iOS/Android, console) are dumb clients joining the same room - one agent binary serves all.

Tech stack. Framework: LiveKit Agents 1.6.10 (Python). Voice: Gemini Live realtime (STT/reasoning/TTS inside one model). Turn detection: LiveKit Inference hosted model. Noise: AiCoustics plugin. Tools: LiveKit `@function_tool` + Playwright Chromium. Transport: LiveKit Cloud rooms (WebRTC); self-hostable server possible (loses hosted Inference + noise cancellation). Vision: room video input -> Gemini multimodal context. Repo: ~3 commits, created 2026-09-07, ~32 stars - tutorial scaffolding, not a maintained product.

Strengths. A single Python process = real-time voice agent with tool use, camera vision, and browser control; LiveKit handles all realtime plumbing; one agent serves web + mobile + terminal. This is the correct architecture for conversational voice and how the official LiveKit starter ecosystem is designed.

Limitations. "100% FREE" is free-tier only: Gemini realtime tokens, LiveKit bandwidth/egress + agent minutes, Inference calls all bill at scale. Browser tools run synchronously in the voice loop - Playwright page loads of several seconds stall the conversation. iPhone support is a dev build (Xcode signing), not App Store deployment. Prerequisites are real: Python ≥3.10, uv, Node 20, pnpm, Flutter SDK, Playwright/Chromium, API keys.

Profile 12 - Stanford OpenJarvis: local-first agent framework

Evidence: architecture VERIFIED (README, docs, arXiv:2605.17172) · benchmark numbers VERIFIED (paper abstract) · press counts CLAIMED

End-to-end workflow. OpenJarvis is a *framework*, not an assistant: a personal-AI system expressed as a typed **spec serialized to portable TOML** over five primitives - Intelligence, Engine, Agents, Tools & Memory, Learning. (1) Entry: CLI (`jarvis ask ...`), Python SDK, browser app (localhost:5173), or Tauri desktop app. (2) Intelligence: pick a model from the local catalog (Qwen3.5, Gemma4, Nemotron, Granite families) or let hardware auto-detection choose. (3) Engine: one `InferenceEngine` interface over Ollama, vLLM, SGLang, llama.cpp, MLX, Exo, LiteLLM, or cloud APIs - auto-selects by GPU vendor/VRAM. (4) Agents: 8 built-in (3 execution modes: on-demand, scheduled, continuous) - `simple`, `orchestrator`, `native_react` (ReAct), `native_openhands` (CodeAct), `deep_research`, `morning_digest`, `monitor_operative`, `operative`. (5) Tools & Memory: built-in tools (web search, calculator, file I/O, code interpreter), native MCP servers, 25+ connectors, 32+ channels, **semantic indexing** (`jarvis memory index ./docs/`, `jarvis memory search "..."`); skills follow the agentskills.io standard and import from Hermes (~150) and OpenClaw (~13,700). (6) Learning: every interaction emits traces (GPU power, token cost, latency); optimize via LoRA, DSPy, GEPA, or **LLM-guided spec search** - a frontier cloud *teacher* reads failure traces, proposes typed spec edits, a held-out gate accepts only non-regressing edits, and the optimized spec runs entirely on-device. (7) Voice: `jarvis chat --voice` (PR #539, ~Aug 2026) - a **blocking turn-based loop** (record-until-silence -> faster-whisper/OpenAI/Deepgram STT -> agent -> kokoro/OpenAI/Cartesia TTS -> playback). Half-duplex; text-first with TTS bolted on, *not* realtime streaming.

Tech stack. Python framework, Apache 2.0, 10k stars, 1,156 commits. Local-first inference; `jarvis serve` exposes an OpenAI-compatible FastAPI+SSE API; Docker/GPU containers, systemd/launchd, desktop releases. OAuth connectors (`jarvis connect gdrive` -> Gmail/Calendar/Tasks) re-introduce cloud data flows where opted in.

Strengths. The core thesis is real and paper-backed: swapping local models into existing stacks loses 25-39pp, but LLM-guided spec search closes the gap - optimized on-device specs match/exceed cloud accuracy on 4 of 8 benchmarks (PinchBench, GAIA), within 3.2pp on average, ~800x lower marginal API cost, 4x lower latency (per paper abstract). Energy/cost/latency as first-class eval axes is genuinely novel. This is the strongest *design philosophy* for the local tier of the Ultimate.

Limitations. Voice is a bolt-on half-duplex loop - anyone expecting LiveKit-grade barge-in will be disappointed. Research-grade maturity (1.0.4.dev, framework released ~Mar 2026). The learning leg depends on a frontier cloud teacher at search time - local-first ≠ cloud-free end to end. 24/7 scheduled agents on a personal GPU have real power/heat cost. (Note: unrelated forks named "openjarvis" describing Discord/whisper/Piper bots are NOT this project.)

Profile 13 - "The Open-Source JARVIS Nobody Is Talking About" (video build on OpenJarvis)

Evidence: base framework VERIFIED · creator's voice/phone layer CLAIMED (unpublished edits) · several video claims GLOSSED

End-to-end workflow. The video's system = Stanford OpenJarvis (Profile 12) as the base + the creator's own unpublished customization. Base flow: text query -> `~/.openjarvis/config.toml` -> engine discovery (default Ollama `qwen3:8b` local) -> router policy picks model (local Qwen 3 8B default; cloud engines - OpenAI/Anthropic/OpenRouter/MiniMax/Gemini - only with API keys or routing) -> memory read via `_get_memory_backend()` -> default **SQLite/FTS5** (`~/.openjarvis/memory.db`), BM25 top_k=5, chunks injected with source attribution; persona files `SOUL.md` / `MEMORY.md` / `USER.md` as system prompt; connector docs (Obsidian vault, Gmail) in the same KnowledgeStore (512-tok chunks, optional ColBERTv2 rerank) -> **OrchestratorAgent** multi-turn tool loop (generate -> tool_calls -> ToolExecutor -> loop, events on EventBus) -> tools: calculator, web_search (Tavily), memory_store/search, file I/O, shell, code execution, knowledge_search, digest TTS -> memory write + **TelemetryRecord per inference** (model, tokens, latency, ttft, cost_usd, energy_joules -> `telemetry.db`) - the "live cost tracking." Messaging: iMessage connector + Gmail ingest; ChannelBridge + Twilio/BlueBubbles/WhatsApp plugins for replies (nontrivial webhook setup, not a toggle). The creator's layer on top: an always-listening desktop voice UI + phone bridge - **no STT/wake-word component is named in any source, and the docs nav has no voice-input page**; an independent setup guide for this exact repo warns "there is no finished wake-word assistant loop." The creator himself: *"I cloned the base and tailored it to my business."*

Tech stack. Local: Qwen 3 8B via Ollama (~6GB VRAM); Apple Silicon/CPU OK. Cloud: OpenAI/Anthropic/Gemini via env keys. Memory: SQLite/FTS5 default; optional faiss/colbert/bm25/hybrid. Connectors (VERIFIED): Gmail/Calendar/Drive/Tasks (one OAuth), Obsidian, Apple Notes, iMessage, Slack, Notion, WhatsApp, GitHub, **Apple Health** (HealthKit DB or iPhone XML export), Oura, Strava, weather, news. TTS: Cartesia/OpenAI/Kokoro - digest-only (`jarvis digest --fresh` cron). Deploy: one-liner install, Docker Compose, Tauri app, `jarvis serve` OpenAI-compatible API. \$0/mo local-only; cloud tokens/TTS/Tavily optional.

Strengths. Closest single match to the "one brain, many surfaces" vision with VERIFIED local-first engineering: hardware-aware engine selection, per-request energy/cost telemetry, data-boundary scan feature, 0600-permission credential store. Connector breadth (including Apple Health) is real.

Limitations. "Always-listening wake word" is the creator's unpublished edit - not in the base. "100% private" only in local-only config; the project's own docs document the cloud data boundary (Cartesia/OpenAI TTS, cloud engines, Gmail OAuth, Tavily). "Codex delegation" - no Codex engine exists. Tailscale remote = his own networking, not a project feature. Caution: many unrelated GitHub projects are also named "OpenJarvis" - only `open-jarvis/OpenJarvis` is the Stanford one.

Profile 14 - eadmin2: Hermes voice + holographic HUD command center

Evidence: VERIFIED (repo `eadmin2/jarvis_ai`, MIT, ARCHITECTURE.md + SETUP.md read) · honest about cloud dependencies

End-to-end workflow. (1) Browser HUD at `https://host/hud/` (self-signed TLS) - click ring -> mic captures 16 kHz mono int16 PCM -> WebSocket `/ws`; or native push-to-talk client on :8765 (wake-word capable via openwakeword). (2) Server: **silero-vad** + **RealtimeSTT** wrapper; incremental **faster-whisper** passes every ~1.2s = live captions; on stop: optional GPU sidecar first (`large-v3-turbo` on LAN NVIDIA box, ~0.2s) -> auto-fallback to local CPU Whisper (`small.en`). (3) Transcript -> **Hermes Agent Sessions API** (`POST /api/sessions/{id}/chat/stream`, SSE); session IDs persist in `logs/hermes_sessions.json` - typed chat uses the *same* session: one persistent Hermes session shared by voice + text. (4) Hermes executes (terminal, files, web search, cron, skills); server parses SSE: `run.started` -> STOP button, `tool.started` -> live HUD activity + command preview, `approval_decision {allow|deny}` over WS -> turn resumes/aborts. (5) Reply sentence-split, markdown/think-blocks stripped, **secret-shaped strings redacted** -> each sentence streams through **ElevenLabs Flash** -> raw PCM over the same WebSocket while generation continues. Measured simple-turn latency ~4s (STT finalize ~0.7s + first token 1-3s + first audio ~3s). (6) Barge-in: cancel-turn protocol that rewrites the agent's memory to match what the user actually heard. (7) Extras: `/api/summon` holographic media panels (bundled `hud_display` Hermes plugin), TLS reverse proxy embedding Hermes kanban + session browser, `/api/machines` host stats. LAN-only by design (docs: don't port-forward).

Tech stack. LLM: BYO - whatever Hermes Agent v0.16 is configured with (Nous Portal 200+ models; free tier = free models; paid \$20-200/mo). STT: faster-whisper local + optional LAN GPU sidecar. TTS: ElevenLabs Flash streaming (~0.5 credits/char; free tier works). Memory: Hermes persistent sessions. Hosting: FastAPI/uvicorn :443 + :8765, Hermes :8642 loopback-only, launchd autostart (macOS tested; Linux "should work").

Strengths. The most *honest* voice build in the set: ARCHITECTURE.md documents 11 hard-won gotchas (SSE mojibake, launchd FD limits, orphaned STT children stealing ports); security thought through (Hermes API key never reaches browser, token-gated WS, loopback dashboard); secret redaction before cloud TTS; approval cards as first-class protocol events; usage tracking (tokens/day + ElevenLabs quota). Best transferable voice blueprint: local STT -> shared persistent session -> streaming TTS -> approval-gated tools.

Limitations. Repo is small (170 stars, 7 commits, one maintainer, macOS-only tested). "3-5s round trip" is simple turns on Apple Silicon; tool-heavy turns slower. "80+ skills" = Hermes' auto-generated skill system, count unverified. "Everything runs on your own hardware" - the repo is upfront: LLM provider + ElevenLabs are cloud. No wake-word loop in the HUD itself (click-to-talk; PTT client optional).

Profile 15 - "Donna" / CortexOS (@agentic.james): Telegram-driven life-OS

Evidence: creator's captions VERIFIED · no public code - template paywalled (\$77/mo) · integration code INFERRED

End-to-end workflow (from his captions + reel narration). "Donna" is his named assistant inside a system he calls CortexOS (also spelled "cortextOS"). Dashboard modules: Calendar, Meeting Transcripts, CRM, Gmail, Approvals, Tasks, iMessage, Workflow Tracking - all driven from a single Telegram chat instead of eight apps. The orchestration layer, per his own captions: "a shared database for tasks and messages," an `agents.md` file "that loads identity, your profile, and guardrails into every agent the second it boots," "a heartbeat to keep them alive," "a shared vector knowledge base every agent can search," and "skills that teach each agent how to use all of it," with "Telegram polling" so the whole thing runs from his phone. As of the reel: 3 Codex agents + 8 Claude Code agents (another reel: 11 agents across Claude Code, Codex, OpenCode) doing content posting, PR reviews, business metrics - coordinated via voice messages from his phone. Donna can schedule workflows, talk to other agents, and draft messages for approval/denial across messaging apps.

Tech stack. Agents: Claude Code + Codex + OpenCode (11 agents). Memory: shared DB + shared vector knowledge base (per his caption - exact store unnamed). Identity: `agents.md` boot file. Transport: Telegram polling (INFERRED: long-polling bot). Runs on his own computer back home. Distribution: "full agent template with custom onboarding" inside the paid Skool community (\$77/mo), 30+ custom skills, MCPs, hooks, subagents; "100+ members" CLAIMED.

Strengths. The `agents.md` -as-boot-identity pattern is genuinely good engineering - identity/profile/guardrails injected into every agent at boot is exactly how you keep multi-agent systems coherent. Shared task DB + shared vector KB across heterogeneous agents (Claude Code + Codex + OpenCode) is the right multi-agent memory topology. Best Telegram-first life-OS demo in the discovery set.

Limitations. No public repo or site - the template is paywalled, so nothing is independently verifiable; stack claims rest on his captions. Exact Telegram integration code undocumented. \$77/mo for the template + the underlying model costs.

Profile 16 - Qendresa Hoti: "Life Admin Brain" local RAG

Evidence: VERIFIED (repo README read: [kju4q/ai-weekend-builds](#) , vol-3/01-life-admin-brain)

End-to-end workflow. The reference implementation of local document RAG, shipped with working code: (1) **Ingestion:** `ingest.py` reads `docs/` - PDFs via `pypdf` , `.txt` , `.md` . (2) **Chunking:** ~800-char chunks - the README calls oversized chunks the #1 pitfall ("one 5-page chunk retrieves everything and answers nothing"). (3) **Embeddings:** `embeddings.py` auto-selects `sentence-transformers` (all-MiniLM-L6-v2) if installed, else a pure-Python hashing-embedding fallback so it runs with zero installs. (4) **Vector store:** a flat `index.json` - cosine similarity, top-4 chunks, all local. (5) **Generation:** if `ANTHROPIC_API_KEY` is set, Claude writes a natural-language answer citing the source file; otherwise local mode just shows retrieved passages. **Only the retrieved chunks for one question ever hit the API** - documents and index never leave the machine. Run: `python ingest.py` , then `python ask.py "when does my passport expire?"` - ships with fake sample docs (lease, passport, insurance, warranty) so it works immediately; swap in real docs and re-ingest. Suggested extensions: Chroma/LanceDB/sqlite-vss, `--source` filter, watch-mode re-ingest.

Tech stack. Python 3.10+, `sentence-transformers` (optional), `pypdf` (optional), Anthropic API (optional). \$0 local; pay-per-token only when Claude answers.

Strengths. The cleanest-documented RAG pattern in the whole corpus: chunk-size discipline, zero-install fallback, and the privacy architecture exactly right (local retrieval, cloud only sees retrieved chunks). Commenters' privacy questions ("Passport id handed to AI?") were answered by the architecture itself. Directly reusable as the Ultimate's document-memory tier.

Limitations. Single-user scripts, no agent loop, no memory writes - it's a retrieval tool, not a system. Flat `index.json` won't scale past thousands of docs (README itself suggests Chroma/LanceDB). Repo link was DM-gated ("Comment BUILD"), found via fork chain.

Profile 17 - memU: wiki-structured agent memory

Evidence: VERIFIED (README + SKILL.md read: NevaMind-AI/memU) · ~14.4k stars · trending claims are README statements

End-to-end workflow. "Personal memory, stored as Wiki. Across Sessions. Across Agents. Across Devices." Not a dumb vector store: memory is organized as human-readable **Markdown Wiki** with a memory track and a skill track, in a shared store (`~/.memu/memu.sqlite3` default; Postgres+pgvector for concurrent/larger deployments). Two seams per host adapter: **record** - a scheduled bridging task (default hourly) slices the agent's session logs (e.g. `~/.claude/projects/<project>/<session>.jsonl` for Claude Code, `~/.codex/sessions/**` for Codex) into self-contained "self-evolve jobs"; the agent itself distills them into memory/skill Markdown; **commit** submits the files. **Inject** - a standing instruction patched into the host's instruction file (e.g. `~/.claude/CLAUDE.md`) tells the agent to run `<binary> retrieve` (`progressive_retrieve`) before answering. Host binaries: `memu-codex` , `memu-claude-code` , `memu-cursor` , `memu-openclaw` , `memu-hermes` , `memu-workbuddy` , `memu-cola` , `memu-pi` , generic `memu-agent` with auto-detect. Embeddings pluggable (OpenAI default; Jina, Voyage, Doubao, OpenRouter). Crucially, **the service makes no LLM calls itself** - judgment stays in the agent; memU only stores/embeds/retrieves. ~500 lines of core logic (Python 3.11+).

Tech stack. SQLite/Postgres+pgvector; pluggable embedding providers; per-host adapter binaries; optional cloud via memu.so API key, self-hosted = local-only. License: README badge says Apache-2.0 (GitHub license field shows NOASSERTION - mild discrepancy noted).

Strengths. The missing piece most builds hand-roll badly: cross-session, cross-agent, cross-device durable memory with *agent-curated* content - the automatic skill-extraction loop (capture -> slice -> agent decides keep/patch/create -> write skill -> commit + embed -> retrieve later) is the procedural-memory tier of the Ultimate, already built. Human-readable Markdown Wiki = debuggable, portable, survives vendor changes.

Limitations. "500 lines of core logic" and trending-#1 are README claims, not independently audited. Hourly batch bridging = memory is eventually consistent, not realtime. On-disk wiki layout couldn't be verified (architecture diagram is an image).

Profile 18 - OpenClaw: self-hosted gateway assistant

Evidence: VERIFIED (README read: `openclaw/openclaw`, MIT)

End-to-end workflow. "The AI that really does things. Any OS. Any Platform." Built around a local **Gateway** - the control plane for sessions, tools, events, and channel connections. Surfaces: Control UI, CLI, TUI, plus native companion apps (macOS/iOS/Android/Windows/Linux) adding voice, camera, screen, and device-local actions. **Messaging channels:** WhatsApp, Telegram, Slack, Discord, Google Chat, Signal, iMessage, 20+ more. Models and agent harnesses (Claude, Codex, local models) are swappable plugins; tools/skills/plugins extend it, shared via ClawHub. Memory: "State, memory, and credentials live on your hardware" - notably, the README doesn't detail a native memory subsystem (memU lists OpenClaw as a supported host, implying memory is often bolted on externally - the memU + OpenClaw pairing is the honest stack). Security model: treat inbound messages as untrusted; DM channels use pairing codes; tools run on host unless sandboxed. Install: `curl -fsSL https://openclaw.ai/install.sh | bash`, then `openclaw onboard --install-daemon`. Stewarded by the OpenClaw Foundation (501(c)(3)).

Tech stack. Self-hosted gateway; WhatsApp/Telegram/etc. channels; swappable model plugins; ClawHub skill marketplace. Cron/scheduling implied via sessions/tools/events (not explicitly verified in README).

Strengths. The broadest channel support of any runtime (WhatsApp native - directly relevant to Naim), companion apps with device-local actions (camera, screen), and a plugin model that keeps models swappable. The Gateway-as-control-plane is the right topology for "one brain, many surfaces."

Limitations. README doesn't document a native memory subsystem - pair with memU or equivalent. Scheduled workflows implied but not README-verified. ~390k stars is the README's figure; treat community scale as directional.

Excluded with reasons (not profiled)

- **@blakeestephens driving call** - pure course funnel; zero architecture ever shown; the banking plot is implausible (banks don't clear fraud freezes on parallel AI phone calls). Keep the closed-loop operator *framing*; discard everything else.
- **Ghost X1 (@realrobertgutierrez)** - "abliterated" is a real technique (VERIFIED: weight-level refusal removal, e.g. huihui-ai models), but "Ghost X1" has zero web presence: no weights, no repo, no docs. Premise is wrong for JARVIS: abliterated models are weaker open fine-tunes - worse at instruction-following, tool use, and long-horizon agency, the exact capabilities needed - and removing the refusal boundary on an always-on agent holding your email, money, and accounts is a liability, not a feature.
- **SouvikChai Python/ML** - repo read file-by-file: dead (0 stars), internally branded "JASMINE" (copied/renamed), toy 27-class Keras classifier with canned random responses, Google cloud STT despite "from scratch" claims. Teaches nothing; anti-pattern only.

PHASE 2 - The Ultimate JARVIS synthesis

1. Synthesized architecture blueprint

The control loop

Every interaction - a WhatsApp message, a wake word, a cron tick, a calendar event - enters the same loop:

```
TRIGGER -> ROUTE -> RECALL -> PLAN -> ACT -> OBSERVE -> VERIFY -> REMEMBER -> RESPOND
```

1. **TRIGGER.** User message (WhatsApp/voice/text), schedule (cron), or external event (calendar, email arrival, alert).
2. **ROUTE (fast, cheap).** A small local model - or deterministic classifier - triages in milliseconds: is this a quick answer (route to cheap model), a deep task (route to frontier reasoning model), or a voice turn (route to realtime voice stack)? *This eliminates the core redundancy of the builds: running a frontier model for everything.* Zubair's hot-swap and Hermes's model-agnostic design prove the brain is config; the router makes it economical.
3. **RECALL.** Before reasoning, pull context: episodic session memory (SQLite + FTS5, Hermes pattern), semantic vault (markdown/Obsidian graph + embeddings, Alex pattern), and procedural skills (SKILL.md files the agent wrote itself). Retrieve by recency × relevance × authority - never dump the whole history into the prompt.
4. **PLAN.** Frontier model produces a plan: steps, tools, and for each step whether it is Tier A (automatic: read/sort/summarize), Tier B (prepare then ask: drafts, calendar proposals), or Tier C (protected: money, messages to others, access changes - explicit approval).
5. **ACT.** Tools execute through one typed tool layer (MCP servers + built-in agent tools + n8n webhooks where visual integration wins). Every external action carries an idempotency key.
6. **OBSERVE.** Read the tool's actual return. Did the draft save? Did the metric load? Did the calendar write conflict?
7. **VERIFY.** Evidence, not confidence: a send is complete only when the API confirms it. Retry with backoff on transient failure; escalate ambiguity to the user instead of guessing.
8. **REMEMBER.** Write back durable facts (decisions, commitments, new procedures as SKILL.md) to the vault/session store.
9. **RESPOND.** Answer on the surface the trigger came from. Proactive outputs (morning brief) go to WhatsApp; missed voice calls fall back to the identical brief as text.

The layered stack (diagram: surfaces thin, brain singular)

```
SURFACES (thin clients, one identity/memory)
  WhatsApp (primary) · Telegram · voice calls · web dashboard
  desktop overlay · watch pings · Android app

      |
VOICE PIPELINE      |      BRAIN = ROUTER + REASONING
  openWakeWord      |      Router: small local model
  faster-whisper (local) -----+-----> Reasoner: frontier hosted model
  Piper/Kokoro (local)  |      Fallback: local Ollama
  ElevenLabs realtime  |      |
  Retell telephony     |      v
      |                |
      |      AGENT RUNTIME
      |      cron · delegation · compression
      |      approvals · audit · idempotency
      |      |
      |      v
      |      MEMORY - THE MOAT
      |      capture · session · procedural
      |      semantic retrieval over vault
      |      |
      |      v
      +-----> TOOLS / HANDS
                  MCP · built-ins · n8n · OS · IoT

HOSTING & PRIVACY: ~$5/mo VPS now · home server later
Local: transcription, memory, sensitive notes
Hosted: frontier reasoning over retrieved context only
Cross-cutting: scheduler · approvals · audit · verification
```

Four architectural decisions that matter most

D1 - Two-speed brain (router + reasoner). No build did this explicitly, and all of them paid for it: either frontier tokens burned on trivia (Zubair) or weak local models fumbled real tasks (manujademin). A millisecond router sending quick answers to a cheap model and deep work to the frontier cuts cost and latency more than any single model upgrade.

D2 - Memory is three separate systems, not one. Capture (Alex's voice-note -> Whisper -> vault habit), session recall (Hermes's SQLite + FTS5), and procedural learning (SKILL.md). Builds that conflated these either forgot everything (Nate) or couldn't find anything (file-dump vaults). Each tier has a different write path, retention rule, and retrieval method.

D3 - Approval tiers + idempotency + verification are the trust architecture. Tier A runs and logs; Tier B prepares and asks; Tier C needs explicit approval. Every external action idempotent; every completion evidenced by the tool's own return. This is what separates an operator from a demo - and it's the boring part no reel shows.

D4 - Surfaces are thin clients; the brain is singular. WhatsApp today, voice calls tomorrow, watch later - all reading the same memory, permissions, and job history (Arcus's vision, Hermes's gateways). Never build a second assistant with a second memory.

2. Consolidated tech stack table

Layer / Component	Recommended Tool / Model	Source System Borrowed From	Technical Justification
Wake word	openWakeWord	manujademin	Millisecond CPU detection, offline, free; no cloud audio streaming
STT (local)	faster-whisper	manujademin, Alex (LocalWhisper)	Best accuracy-per-watt offline; keeps intimate audio off clouds
STT (realtime)	ElevenLabs Conversational AI built-in	Nate	Turn-taking + interruption handling you can't hand-roll cheaply
TTS (local)	Piper TTS / Kokoro-82M	manujademin (Piper), Chloe (Kokoro)	Kokoro: 82M params, Apache 2.0, CPU-runnable, free; Piper: mature ONNX voices
TTS (realtime)	ElevenLabs	Nate, Hermes tutorial	Natural prosody; metered - reserve for conversations that earn it
Router model	Small local model (e.g. Qwen 3 1-4B via Ollama) or classifier	Synthesis (no build did this)	ms triage; stops frontier-token burn on trivia
Reasoning model	Frontier hosted, hot-swappable (Claude Fable 5.x / GPT / Grok via endpoint switch)	Zubair (hot-swap), Hermes (model-agnostic), Azaris (BYOK)	Model is config, not moat; 1M-token context for big briefs
Fallback model	Ollama local (Llama/Qwen 8B)	manujademin	Offline + fully private tasks; zero marginal cost
Agent runtime / harness	Hermes Agent (MIT) or OpenClaw (MIT)	Hermes tutorial, eadmin2, OpenClaw	Genuine agents: tools + gateways + sessions; OpenClaw adds native WhatsApp + companion apps; verify Hermes memory writes
Realtime voice framework	LiveKit Agents 1.6.10 + Gemini Live	Thanhy (ruxakK)	WebRTC rooms, server VAD, adaptive barge-in, function tools; one agent serves web + mobile + console
Episodic memory	SQLite + FTS5 session store	Hermes	Full-text search across all conversations; simple, local, fast
Semantic memory	Markdown/Obsidian vault + embeddings (local)	Alex	Human-readable, portable, survives vendor changes; add vector index at scale
Procedural memory	SKILL.md files written by the agent	Hermes learning loop	The system improves with use instead of re-learning
RAG over documents	Local RAG pipeline (chunk -> embed -> vector store -> retrieve)	Qendresa "Life Admin Brain"	For leases/passports/manuals: deterministic retrieval beats hoping the model remembers

Layer / Component	Recommended Tool / Model	Source System Borrowed From	Technical Justification
Agent shared memory	memU (SQLite/Postgres+pgvector, Markdown wiki)	memU (VERIFIED)	Cross-session/agent/device memory the agent curates; hourly bridge; no LLM calls in service
Tools / MCP layer	MCP servers (Playwright, business-data) + built-in tools	Chloe	MCP is the adapter standard; Playwright for browser, RevenueCat/Meta-style for metrics
Visual integrations	n8n webhooks (self-hosted when serious)	Nate	400+ integrations without hand-written code; self-host to avoid per-execution caps
Scheduler	Cron in agent runtime + overnight batch jobs	Hermes, Azaris pattern	Jobs with timeouts/retries; "prepare vs send" separation
Approval gating	Approval cards, Tier A/B/C	Azaris, eadmin2	Money/messages/access changes never auto-execute
Audit & idempotency	Append-only action log; idempotency keys on external actions	Synthesis (Luke's pattern done right)	Retries must not double-send; every finish evidenced
Messaging surfaces	WhatsApp gateway first, then Telegram	Hermes gateways, Donna, Denny Santoso	Where Naim already lives; proactive briefs land as messages
Voice surface	Realtime voice agent; telephony via Retell-style (\$0.11-0.16/min)	Nate, Zubair	Calls only when a concrete use case earns the meter
Dashboard	Claude-generated web UI / Hermes dashboard	Chloe, Zubair	Review + approvals surface; generate, don't hand-build
Hosting (now)	~\$5/mo VPS, always-on	Hermes tutorial, Velvet Shark	A sleeping laptop is not a server
Hosting (later)	Home mini-PC / Mac Mini-class server	Alex, Arcus	24/7 runtime + local Ollama fallback; Phase 3 only
Vision	Multimodal model frames (sparingly)	Zubair	Token-hungry - gate behind the router, not always-on
Privacy tiering	Local: transcription + memory store. Hosted: reasoning over retrieved context	manujademin, Alex, Arcus	Every real build converged here: local capture, hosted brain

3. End-to-end execution trace

Command (via WhatsApp text): *"Check my unread emails, extract any travel dates, book a calendar slot, and dim the office lights."*

1. **TRIGGER.** WhatsApp gateway (Hermes) receives the message; creates a job record: goal, trigger=user, timestamp.
2. **ROUTE.** Router (small local model): multi-step task, needs email + calendar + IoT tools -> frontier reasoning model. Not a voice turn, not trivia.

3. **RECALL.** Session memory: has Naim booked travel before? Vault: "office lights" -> Home Assistant entity `light.office` (learned SKILL.md from prior setup). Prayer times for today (scheduling constraint - never book over a prayer).
4. **PLAN (frontier).** Steps: (a) Gmail: list unread [Tier A]; (b) parse travel dates from results [Tier A]; (c) find free calendar slot avoiding conflicts + prayers, *propose* [Tier B - booking needs approval]; (d) dim lights [Tier A - reversible, low-risk].
5. **ACT.**
 - Tool `gmail.list(unread, max=20)` -> returns 7 unread. Tool `gmail.get` on 3 with travel keywords -> bodies returned.
 - LLM parse: "flight to Chicago Oct 14, return Oct 17" extracted with confidence + source message IDs.
 - Tool `calendar.freebusy(Oct 14-17)` -> Oct 15 2-4 PM free; but Asr prayer ~3:18 PM -> slot adjusted to 1:30-2:30 PM proposed. **Tier B: approval card sent to WhatsApp** - "Book 'Trip prep' Oct 15, 1:30-2:30 PM?" Naim taps approve -> `calendar.create` -> **API returns event ID** -> verified.
 - Tool `homeassistant.light.turn_on(entity=light.office, brightness_pct=30)` -> returns new state `brightness: 76/255` -> verified dimmed.
6. **OBSERVE/VERIFY.** Each tool's return checked: Gmail returned (not cached), event ID exists, light state confirms. If Gmail API had failed: retry with backoff x2, then escalate ("couldn't reach Gmail - try again or check manually") rather than inventing emails. If the date parse were ambiguous ("next Friday" vs "Friday"), Tier B: ask. If the light were unreachable: report partial completion honestly - "emails checked, slot booked, but the office light didn't respond."
7. **REMEMBER.** Vault write: "Travel: Chicago Oct 14-17 (from email <id>). 'Trip prep' booked Oct 15 1:30-2:30 PM. Office light entity confirmed working."
8. **RESPOND (WhatsApp).** "Checked 7 unread - found one travel item: Chicago Oct 14-17. Booked 'Trip prep' Oct 15, 1:30-2:30 PM (moved clear of Asr). Office lights dimmed to 30%."

Failure branches, plainly: tool timeout -> retry -> escalate; ambiguous date -> ask, don't guess; calendar conflict -> propose 2 alternatives; approval ignored for 15 min -> hold the booking, complete the rest, note it; any Tier C accidentally triggered (e.g., an email asking to wire money) -> hard stop, surface to Naim.

4. Build recommendation for Naim

Calibrated to: non-programmer building with AI coding tools; ~\$5-10/mo Phase 1 budget (every dollar weighed against debt recovery); WhatsApp-first; prayer times and the 5:30 AM-12:30 PM sleep block as hard scheduling constraints; Amazon FBA + Uber as the business surface.

Phase 1 - Weekend V1: "The Remembering Assistant" (~\$5-10/mo). Hermes-class runtime (MIT, free) on a ~\$5/mo VPS - a sleeping laptop is not a server. WhatsApp interface via the messaging gateway. Memory: markdown vault + session store; the one habit that matters: **2-minute end-of-day voice note** ->

local Whisper -> vault (Alex's pipeline at 1% of her hardware). Two tools only: Gmail + Calendar read (already connected) + web search. Scheduler: prayer-aware morning brief to WhatsApp daily (never scheduled over a prayer), nightly shutdown note. Model: OpenRouter free tier to start (50 req/day unfunded), frontier when token burn justifies it. **Done** = it references something from 3 weeks ago unprompted; brief arrives daily for 30 days.

Phase 2 - Voice + money tools (~\$25-50/mo, only when Phase 1 is boring). ElevenLabs realtime voice or local Whisper+Piper/Kokoro to stay near \$0. Money tools: Chloe's MCP loop pointed at *his* business (Amazon Seller Central data) - **approval cards before anything spends or messages**. Agent starts writing its own SKILL.md files.

Phase 3 - The operator (one-time ~\$100-200 hardware). Home mini-PC: 24/7 runtime + local Ollama fallback. More surfaces (desktop overlay, watch pings, web dashboard). Subagents (Research/Developer/Browser) only after the single agent is reliable - multi-agent multiplies failure modes too.

V1 vs. later, plainly. V1 = memory + briefs + two tools on WhatsApp. Voice calls, screen control, business automation, multi-agent, local-LLM hardware are all later. **Do not buy hardware in Phase 1.**

What NOT to build/copy. Course/lead funnels (Blake, Z.E.R.O consulting, Ghost X1 keyword gates); the Ghost X1 uncensored-model premise; toy classifiers; cargo-cult brain-UIs and "32 tools" without idempotent tools, approval gates, memory hygiene, eval loops; memory-less SaaS voice chains as the brain (Nate's stack is a mouth, not a mind); "fully local" as a purity test; scripted ads as evidence.

The single highest-leverage first step. Get an always-on agent runtime with persistent memory and a WhatsApp interface briefing him every morning - before spending anything on voice realism, dashboards, or hardware. Across every build, the differentiator was never the model; it was what the system remembers and what it's allowed to touch.

Open verification items / honest caveats

- YouTube pages largely resist scraping; some mechanics were reconstructed from descriptions, repos, and component docs - labeled INFERRED throughout.
- "Fletcher" (Alex's retrieval layer) internals are unverified - her own naming, no public repo.
- Arcus's "fully local" claim is unverified (closed source); skeptics allege Claude wrapper.
- Zubair's 8 build prompts are paywalled - the "one prompt does the work of three" claim is unverifiable; no independent reproductions found.
- Hermes Agent has VERIFIED fail-silent memory bugs (silent history deletion on failed summarization) - any build on it must verify writes independently.
- Chloe's quoted metrics (\$4,289/\$475 $\approx$ 9x, not the stated 1.5 ROAS) should be treated as illustration.
- Azaris's OpenClaw lineage is suggestive (GitHub org) but unconfirmed; its "persistent memory" needs verification.

- Ghost X1 has zero verifiable existence (no weights, repo, or docs).
- Donna/CortexOS has no public code - its template is paywalled (\$77/mo); architecture claims rest on the creator's captions and reel narration.
- Qendresa's Life Admin Brain repo was found via its fork chain (DM-gated distribution); the RAG code itself was README-verified.
- memU's "500 lines of core logic" and trending-#1 claims are README statements, not independent audits; its GitHub license field shows NOASSERTION vs the README's Apache-2.0 badge.
- OpenClaw's cron/scheduling was not explicitly verified in the README; its native memory subsystem is undocumented (pair with memU or equivalent).
- The OpenJarvis video's "always-listening wake word" and phone bridge are the creator's unpublished edits - absent from the base repo's docs.
- eadmin2's "80+ skills" count is asserted by its README, consistent with Hermes's skill system but not independently counted.

Sources

Primary reels, videos, repos, and docs are linked inline in each profile. Prior reports: [jarvis-reels-deep-dive.pdf](#) (7 reels), [jarvis-builds-deep-dive.pdf](#) (13 builds).